

图搜索强化训练

问题 5-5: 口袋魔方 (Pocket Cube)

涵盖内容:

- ✓ 状态空间与隐式图建模
- ✓ 广度优先搜索与最短路径
- ✓ 魔方状态数量的组合计数
- ✓ 图中顶点度数与可逆操作
- ✓ 双向 BFS 与搜索空间优化
- ✓ Python 路径重建与操作序列恢复

编程语言: Python

请先完成理论分析, 再重写 `solve(config)` 函数

问题 5-1 口袋魔方

考点：隐式状态图 • BFS • 最短路径 • 双向搜索 • 组合计数

1.1 问题背景

口袋魔方（Pocket Cube）是传统 $3 \times 3 \times 3$ 魔方的一个较小变体，其尺寸为

$$2 \times 2 \times 2.$$

它由 8 个角块组成，每个角块有 3 个可见面，并且这三个面具有不同颜色。

当口袋魔方处于复原状态（solved configuration）时，它的每一个 2×2 外表面都是单色的。使用下标集合

$$i \in \{0, \dots, 5\}$$

表示六种颜色，并将第 i 种颜色记为 c_i 。

不失一般性，我们固定其中一个角块的位置和朝向。在此基础上，只允许绕口袋魔方三个表面

$$\{f_0, f_1, f_2\}$$

的法向量进行单次旋转。这三个表面均不包含被固定的角块。

一次操作（move）由二元组

$$(j, s)$$

表示，其中：

- $j \in \{0, 1, 2\}$ 表示旋转表面 f_j ；
- $s = 1$ 表示顺时针旋转一次；
- $s = -1$ 表示逆时针旋转一次。

可以使用广度优先搜索解决口袋魔方问题。为此，将问题表示为一张图：

- 每个顶点表示口袋魔方的一种可能配置；
- 如果一种配置可以通过一次操作变为另一种配置，则在这两个顶点之间连一条边。

不必显式存储整张图的邻接关系。对于任意给定配置，只需对它应用所有可能的单次操作，即可计算出该配置的全部邻居。

1.2 口袋魔方与展开图示意

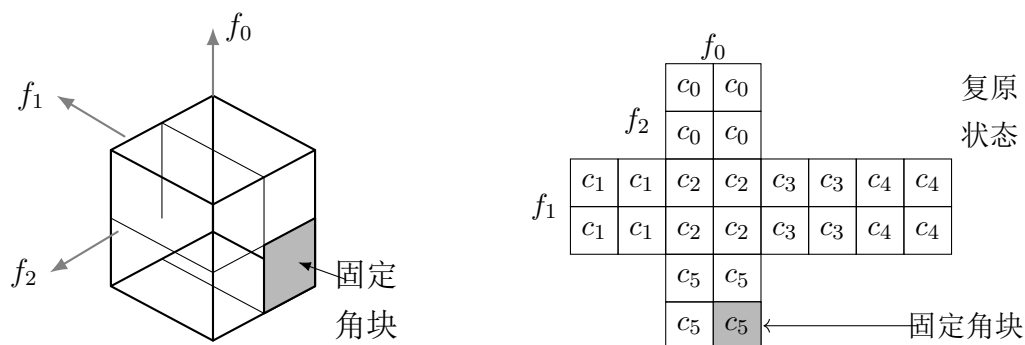


图 1: 固定一个角块后, 允许旋转的三个表面及复原状态展开图

1.3 分问

- (a) 证明口袋魔方不同配置的数量少于 1200 万。请使用组合计数方法, 尽可能给出一个紧的上界。

思考提示: 分别思考剩余角块的排列方式和朝向。固定一个角块后, 还有多少个角块可以重新排列? 每个角块有多少种可能朝向? 所有角块的朝向是否完全相互独立?

- (b) 写出口袋魔方状态图中任意顶点的最大度数和最小度数。

思考提示: 从任意配置出发, 有多少种允许的单次操作? 顺时针与逆时针旋转是否可能得到相同配置? 一个合法操作是否始终存在对应的逆操作?

- (c) 题目提供的模板代码使用广度优先搜索, 从给定配置开始完整探索口袋魔方状态图, 然后返回一个能够复原口袋魔方的操作序列 (假设复原配置可以到达)。

然而, 这个求解器非常慢。¹

运行所提供的代码, 并写出搜索过程访问的配置数量。这个数量与你在第 (a) 问中得到的上界相比如何?

思考提示: 程序结束前会输出 `Searched ... reachable configurations`。记录该数字, 并区分“所有理论排列数”和“从合法状态实际可达的配置数”。

¹该程序可能需要数分钟运行时间, 并需要超过 400 MB 的内存。

- (d) 写出能够复原任意可解口袋魔方所需操作数 w 的最大值。

思考提示：这里的 w 是状态图中任意可达配置到复原配置的最短距离。它也可以理解为这张状态图相对于复原状态的最大 BFS 层数。

- (e) 对某个特定配置，令 N_i 表示从该配置出发，在不超过 i 次操作内可以到达的口袋魔方配置数量。

所提供的代码会访问 N_w 个配置，其数量超过 300 万。

请设计一种算法，对任意口袋魔方配置：

- 找到一个最短复原操作序列；或者
- 当不存在复原序列时，返回“不存在”。

你的算法访问的配置数量不得超过

$$2N_{\lceil w/2 \rceil},$$

该数量少于 9 万。

思考提示：单向 BFS 从起点一直搜索到终点。是否可以同时从初始配置和复原配置开始搜索？当两个搜索前沿相遇时，如何利用两棵父节点树重建完整路径？为保证路径最短，应当如何安排两个方向的搜索层数？

- (f) 根据第 (e) 问设计的算法，重写模板代码中的

`solve(config)`

函数。

思考提示：分隔线以下的辅助函数已经实现，无需修改。你需要维护两个父节点字典和两个搜索前沿，检测两个搜索方向是否出现公共配置，再将两段路径正确拼接。

1.4 提交要求

1. 提交第（a）至第（e）问的文字分析。
2. 提交重写后的 `solve(config)` 函数。
3. 运行模板中的 `test(config)` 验证操作序列。
4. 记录搜索访问的配置数量，并与原始单向 BFS 比较。
5. 除 `solve(config)` 外，不要修改分隔线以下的代码。

附录：Python 代码模板

下面是题目提供的完整代码模板。第（f）问只需重写开头的 `solve(config)` 函数。

```

1  # ----- #
2  # REWRITE SOLVE IMPLEMENTING PART (e) #
3  # ----- #
4  def solve(config):
5      # Return a sequence of moves to solve config,
6      # or None if not possible
7
8      # Fully explore graph using BFS
9      parent, frontier = {config: None}, [config]
10     while len(frontier) != 0:
11         frontier = explore_frontier(frontier, parent, True)
12         print('Searched %s reachable configurations' % len(parent))
13
14     # Check whether solved state visited and reconstruct path
15     if SOLVED in parent:
16         path = path_to_config(SOLVED, parent)
17         return moves_from_path(path)
18     return None
19
20 # ----- #
21 # READ, BUT DO NOT MODIFY CODE BELOW HERE
22 # ----- #
23 # Pocket Cube configurations are represented by length 24 strings
24 # Each character represents the color of a small cube face
25 # Faces are laid out in reading order of a Latin cross unfolding
26 # of the cube
27
28 SOLVED = '000011223344112233445555'
29
30 def config_str(config):
31     # Return config string representation as a Latin cross unfolding
32     return """
33         %s%s
34         %s%s
35     %s%s%s%s%s%s%s%s
36     %s%s%s%s%s%s%s%s

```

```
37         '%s%s'
38         '%s%s'
39         """ % tuple(config)
40
41 def shift(A, d, ps):
42     # Circularly shift values at indices ps in list A by d positions
43     values = [A[p] for p in ps]
44     k = len(ps)
45     for i in range(k):
46         A[ps[i]] = values[(i - d) % k]
47
48 def rotate(config, face, sgn):
49     # Returns new config by rotating input face of input config
50     # Rotation is clockwise if sgn == 1,
51     # counterclockwise if sgn == -1
52     assert face in (0, 1, 2)
53     assert sgn in (-1, 1)
54     if face is None:
55         return config
56
57     new_config = list(config)
58
59     if face == 0:
60         shift(new_config, 1 * sgn, [0, 1, 3, 2])
61         shift(new_config, 2 * sgn, [11, 10, 9, 8, 7, 6, 5, 4])
62
63     elif face == 1:
64         shift(new_config, 1 * sgn, [4, 5, 13, 12])
65         shift(new_config, 2 * sgn, [0, 2, 6, 14, 20, 22, 19, 11])
66
67     elif face == 2:
68         shift(new_config, 1 * sgn, [6, 7, 15, 14])
69         shift(new_config, 2 * sgn, [2, 3, 8, 16, 21, 20, 13, 5])
70
71     return ''.join(new_config)
72
73 def neighbors(config):
74     # Return neighbors of config
75     ns = []
76     for face in (0, 1, 2):
```

```
77     for sgn in (-1, 1):
78         ns.append(rotate(config, face, sgn))
79     return ns
80
81 def explore_frontier(frontier, parent, verbose=False):
82     # Explore frontier, adding new configs to parent and new_frontier
83     # Prints size of frontier if verbose is True
84     if verbose:
85         print(
86             'Exploring next frontier containing # configs: %s'
87             % len(frontier)
88         )
89
90     new_frontier = []
91
92     for f in frontier:
93         for config in neighbors(f):
94             if config not in parent:
95                 parent[config] = f
96                 new_frontier.append(config)
97
98     return new_frontier
99
100 def path_to_config(config, parent):
101     # Return path of configurations from root of parent tree to config
102     path = [config]
103
104     while path[-1] is not None:
105         path.append(parent[path[-1]])
106
107     path.pop()
108     path.reverse()
109     return path
110
111 def moves_from_path(path):
112     # Given path of configurations, return list of moves relating them
113     # Returns None if any adjacent configs on path are not related by a move
114     moves = []
115
116     for i in range(1, len(path)):
```



```
117     move = None
118
119     for face in (0, 1, 2):
120         for sgn in (-1, 1):
121             if rotate(path[i - 1], face, sgn) == path[i]:
122                 move = (face, sgn)
123                 moves.append(move)
124
125     if move is None:
126         return None
127
128     return moves
129
130 def path_from_moves(config, moves):
131     # Return the path of configurations from input config
132     # applying input moves
133     path = [config]
134
135     for move in moves:
136         face, sgn = move
137         config = rotate(config, face, sgn)
138         path.append(config)
139
140     return path
141
142 def scramble(config, n):
143     # Returns new configuration by applying n random moves to config
144     from random import randint
145
146     for _ in range(n):
147         ns = neighbors(config)
148         i = randint(0, 2)
149         config = ns[i]
150
151     return config
152
153 def check(config, moves, verbose=False):
154     # Checks whether applying moves to config results in the solved config
155     if verbose:
156         print(
```

```
157         'Making %s moves from starting configuration:'
158         % len(moves)
159     )
160
161     path = path_from_moves(config, moves)
162
163     if verbose:
164         print(config_str(config))
165
166     for i in range(1, len(path)):
167         face, sgn = moves[i - 1]
168         direction = 'clockwise'
169
170         if sgn == -1:
171             direction = 'counterclockwise'
172
173         if verbose:
174             print('Rotating face %s %s:' % (face, direction))
175             print(config_str(path[i]))
176
177     return path[-1] == SOLVED
178
179 def test(config):
180     print('Solving configuration:')
181     print(config_str(config))
182
183     moves = solve(config)
184
185     if moves is None:
186         print('Path to solved state not found... :(')
187         return
188
189     print('Path to solved state found!')
190
191     if check(config, moves):
192         print('Move sequence terminated at solved state!')
193     else:
194         print('Move sequence did not terminate at solved state... :(')
195
196 if __name__ == '__main__':
```

```
197     config = scramble(SOLVED, 100)
198     test(config)
```

附：实验记录表

项目	实验记录
第 (a) 问得到的配置数上界	
原始 BFS 访问的配置数量	
最大最短复原步数 w	
双向 BFS 访问的配置数量	
返回的复原操作数量	
<code>check(config,moves)</code> 的结果	
运行时间与 Python 版本	

附：编写建议与常见误区

1. **先理解状态图。**一个顶点是一种魔方配置，一条边表示一次合法旋转。程序并没有预先存储整张图，而是通过 `neighbors(config)` 动态生成邻居。
2. **BFS 第一次到达某个状态时得到的是最短路径。**因此，每个父节点字典只应在首次发现配置时写入，不应覆盖已经存在的父节点。
3. **双向搜索需要两个独立的 `visited/parent` 结构。**一个从输入配置出发，另一个从 SOLVED 出发。两个方向的父节点含义不同，拼接路径时需要特别注意方向。
4. **从复原状态一侧获得的路径必须反向。**如果路径是从复原状态到相遇点建立的，那么最终需要把它变成从相遇点到复原状态的顺序。
5. **旋转操作具有逆操作。**操作 (j, s) 的逆操作为

$$(j, -s).$$

这可以用于检查反向路径中的操作方向是否正确。

6. 避免一次性完整探索状态空间。第（e）问的目标就是在搜索深度大致减半的情况下，利用双向 BFS 显著减少需要访问的配置数量。

7. 先测试特殊输入。至少测试以下情况：

- 输入已经是 SOLVED；
- 输入与 SOLVED 相差一步；
- 输入是随机打乱后的合法配置；
- 返回结果能够通过 check 验证。

原题参考

口袋魔方背景介绍：

https://en.wikipedia.org/wiki/Pocket_Cube